

torch.linalg.multi_dot#

https://pytorch.org/docs/stable/generated/torch.linalg.multi_dot.html

Description:

Efficiently multiplies two or more matrices by reordering the multiplications so that the fewest arithmetic operations are performed. Supports inputs of float, double, cfloat and cdouble dtypes. This function does not support batched inputs. Every tensor in tensors must be 2D, except for the first and last which may be 1D. If the first tensor is a 1D vector of shape (n,) it is treated as a row vector of shape (1, n), similarly if the last tensor is a 1D vector of shape (n,) it is treated as a column vector of shape (n, 1). If the first and last tensors are matrices, the output will be a matrix. However, if either is a 1D vector, then the output will be a 1D vector. Differences with `numpy.linalg.multi_dot`:

Function Signature

```
torch.linalg.multi_dot(tensors, *, out=None)#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – output tensor. Ignored if None. Default: None.

Code Examples

```
>>> from torch.linalg import multi_dot

>>> multi_dot([torch.tensor([1, 2]), torch.tensor([2, 3])])
tensor(8)
>>> multi_dot([torch.tensor([[1, 2]]), torch.tensor([2, 3])])
tensor([8])
>>> multi_dot([torch.tensor([[1, 2]]), torch.tensor([[2],
[3]])])
tensor([[8]])

>>> A = torch.arange(2 * 3).view(2, 3)
>>> B = torch.arange(3 * 2).view(3, 2)
>>> C = torch.arange(2 * 2).view(2, 2)
>>> multi_dot((A, B, C))
tensor([[ 26,  49],
        [ 80, 148]])
```

Notes & Warnings

⚠ WARNING This function does not broadcast.

NOTE This function is implemented by chaining `torch.mm()` calls after computing the optimal matrix multiplication order.

NOTE The cost of multiplying two matrices with shapes (a, b) and (b, c) is $a * b * c$. Given matrices A, B, C with shapes $(10, 100), (100, 5), (5, 50)$ respectively, we can calculate the cost of different multiplication orders as follows:

$$\begin{aligned} \text{cost}((AB)C) &= 10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500 \\ \text{cost}(A(BC)) &= 10 \times 100 \times 50 + 100 \times 5 \times 50 = 75000 \end{aligned}$$

In this case, multiplying A and B first followed by C is 10 times faster.

Detailed Documentation

Documentation

Efficiently multiplies two or more matrices by reordering the multiplications so that the

fewest arithmetic operations are performed.

Supports inputs of float, double, cfloat and cdouble dtypes. This function does not support batched inputs.

Every tensor in tensors must be 2D, except for the first and last which may be 1D. If the first tensor is a 1D vector of shape (n,) it is treated as a row vector of shape (1, n), similarly if the last tensor is a 1D vector of shape (n,) it is treated as a column vector of shape (n, 1).

If the first and last tensors are matrices, the output will be a matrix. However, if either is a 1D vector, then the output will be a 1D vector.

Differences with `numpy.linalg.multi_dot`:

Unlike `numpy.linalg.multi_dot`, the first and last tensors must either be 1D or 2D whereas NumPy allows them to be nD

This function does not broadcast.

This function is implemented by chaining `torch.mm()` calls after computing the optimal matrix multiplication order.

The cost of multiplying two matrices with shapes (a, b) and (b, c) is $a * b * c$. Given matrices A, B, C with shapes (10, 100), (100, 5), (5, 50) respectively, we can calculate the cost of different multiplication orders as follows:

In this case, multiplying A and B first followed by C is 10 times faster.

tensors (Sequence[Tensor]) – two or more tensors to multiply. The first and last tensors may be 1D or 2D. Every other tensor must be 2D.

out (Tensor, optional) – output tensor. Ignored if None. Default: None.

